

Machine Learning II Mini Project

Student Name:	Abdullah Alatwi	Student ID:	202221026
Student Name:	khalid Almalki	Student ID:	202332029
Student Name:	Faisal Al-Juhani	Student ID:	202121028

1. Project Title

Integrated Learning Paradigms Using Python

2. Problem Statement

The goal of this project is to apply different machine learning methods on simple data using Python. We want to understand how each method works and compare them.

3. Objective

In this project, we apply 4 types of learning:

- Supervised Learning (SVM)
- Reinforcement Learning (Q-learning)
- Unsupervised Learning (K-means)
- Neural Networks

4. Tools Used

- Python
- scikit-learn
- NumPy

5. Part A – Supervised Learning (SVM)

Explanation

SVM is used for classification. It finds a line that separates the data into classes.

Maximum Margin

The best line is the one that has the biggest distance between the classes.

Code

```
C:\Users\abdul > OneDrive > Documents > from sklearn.py > ...
1  from sklearn.datasets import load_iris
2  from sklearn.model_selection import train_test_split, cross_val_score
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.svm import SVC
5  from sklearn.metrics import accuracy_score, zero_one_loss, mean_absolute_error, mean_squared_error
6
7  iris = load_iris()
8  X = iris.data
9  y = iris.target
10
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
12
13 scaler = StandardScaler()
14 X_train = scaler.fit_transform(X_train)
15 X_test = scaler.transform(X_test)
16
17 model = SVC(kernel='linear', C=1)
18 model.fit(X_train, y_train)
19
20 pred = model.predict(X_test)
21
22 print("Accuracy:", accuracy_score(y_test, pred))
23 print("0-1 Loss:", zero_one_loss(y_test, pred))
24 print("L1 Loss:", mean_absolute_error(y_test, pred))
25 print("L2 Loss:", mean_squared_error(y_test, pred))
26
27 scores = cross_val_score(model, X, y, cv=5)
28 print("CV Average:", scores.mean())
29
```

Regularization

cost = loss + $\lambda \times$ complexity

If C is large $\rightarrow$ model is complex If C is small $\rightarrow$ model is simple

Output:

The model performs well and shows high accuracy on both test data and cross-validation.

```
Accuracy: 0.9555555555555556
0-1 Loss: 0.04444444444444444
L1 Loss: 0.04444444444444446
L2 Loss: 0.04444444444444446
CV Average: 0.9800000000000001
```

6. Part B – Reinforcement Learning

Explanation

The agent learns by trying actions and getting rewards.

Q-learning Rule

$$Q(s,a) = Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

```
C:\Users\abdul > OneDrive > Documents > from sklearn.py > ...
1  import numpy as np
2  import random
3
4  q = np.zeros((6,2))
5  alpha = 0.1
6  gamma = 0.9
7  epsilon = 0.2
8
9  for _ in range(200):
10     state = 0
11     while state != 5:
12         if random.random() < epsilon:
13             action = random.randint(0,1)
14         else:
15             action = np.argmax(q[state])
16
17         if action == 0:
18             next_state = min(state+1,5)
19         else:
20             next_state = max(state-1,0)
21
22         reward = 1 if next_state == 5 else 0
23
24         q[state,action] += alpha * (reward + gamma*np.max(q[next_state]) - q[state,action])
25
26         state = next_state
27
28  print(q)
```

Output:

The agent learned that moving forward gives higher reward, so it chooses better actions over time.

```
[[0.65609864 0.53266521]
 [0.72899972 0.53863114]
 [0.80999995 0.54424249]
 [0.89999999 0.62758055]
 [1.         0.6783364 ]
 [0.         0.         ]]
```

ϵ -greedy

Sometimes we explore, sometimes we choose the best action.

7. Part C – K-means

Explanation

K-means groups similar data points into clusters.

Steps

1. Choose k
2. Assign points to nearest center
3. Update centers (mean)
4. Repeat

```
C: > Users > abdul > OneDrive > Documents > from sklearn.py > ...  
1  from sklearn.cluster import KMeans  
2  from sklearn.datasets import load_iris  
3  
4  iris = load_iris()  
5  X = iris.data  
6  
7  kmeans = KMeans(n_clusters=3)  
8  kmeans.fit(X)  
9  
10 print(kmeans.cluster_centers_)
```

Output:

The cluster centers are updated by taking the mean of the points in each cluster.

```
[[5.88360656 2.74098361 4.38852459 1.43442623]  
 [5.006      3.428      1.462      0.246      ]  
 [6.85384615 3.07692308 5.71538462 2.05384615]]
```

8. Part D – Neural Network

Explanation

Neural network learns patterns from data.

```
C: > Users > abdul > OneDrive > Documents > from sklearn.py > ...  
1 from sklearn.neural_network import MLPClassifier  
2 from sklearn.model_selection import train_test_split  
3 from sklearn.datasets import load_iris  
4 from sklearn.preprocessing import StandardScaler  
5 from sklearn.metrics import accuracy_score  
6  
7 iris = load_iris()  
8 X = iris.data  
9 y = iris.target  
10  
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)  
12  
13 scaler = StandardScaler()  
14 X_train = scaler.fit_transform(X_train)  
15 X_test = scaler.transform(X_test)  
16  
17 model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000, solver='sgd')  
18 model.fit(X_train, y_train)  
19  
20 pred = model.predict(X_test)  
21 print("Accuracy:", accuracy_score(y_test, pred))  
22
```

Output:

The neural network learns patterns using backpropagation to update the weights.

```
Accuracy: 0.9333333333333333
```

Backpropagation

It updates the weights using the error.

9. Conclusion

In this project, we used different machine learning methods. Each method solves a different problem. This helped us understand the basics of machine learning.

10. Final Answer

We applied SVM, Q-learning, K-means, and neural networks using Python to understand different machine learning methods.